

## Sumário

|                                                                                                                                                                 |                                      |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------|
|  Apostila: ArrayList em Java — Métodos Essenciais .....                        | 2                                    |
| 1 add() — Adiciona elementos à lista .....                                                                                                                      | 2                                    |
| 2 get(int index) — Acessa um elemento .....                                                                                                                     | 2                                    |
| 3 set(int index, E elemento) — Altera um item .....                                                                                                             | 2                                    |
| 4 remove(int index) — Remove por posição .....                                                                                                                  | 3                                    |
| 5 size() — Tamanho da lista .....                                                                                                                               | 3                                    |
| 6 clear() — Limpa a lista .....                                                                                                                                 | 3                                    |
| 7 contains(E elemento) — Verifica se existe .....                                                                                                               | 4                                    |
| 8 isEmpty() — Verifica se está vazia .....                                                                                                                      | 4                                    |
| 9 indexOf(E elemento) — Posição de um item .....                                                                                                                | 4                                    |
|  Métodos e recursos adicionais do ArrayList que valem ouro .....               | 4                                    |
|  1. replaceAll() — Substitui todos os elementos com base em uma regra .....    | 4                                    |
|  2. sort() — Ordena a lista .....                                              | 5                                    |
|  3. reverse() — Inverte a ordem da lista .....                                 | 5                                    |
|  4. subList(int fromIndex, int toIndex) — Pega uma parte da lista .....       | 5                                    |
|  5. forEach() — Itera com função lambda .....                                | 6                                    |
|  6. removeIf() — Remove com condição .....                                   | 6                                    |
|  7. equals() — Compara listas .....                                          | 6                                    |
|  8. forEach(System.out::println) — Imprime todos os elementos da lista ..... | 6                                    |
| 📄 o jeito tradicional com for: .....                                                                                                                            | 7                                    |
|  for com índice — o clássico .....                                           | 7                                    |
| DIREFENÇAS ENTRE FOR'S: .....                                                                                                                                   | 8                                    |
| Dicas de nomes não mencionados nessa Apostila: .....                                                                                                            | 8                                    |
|  O que é uma expressão lambda em Java? .....                                 | 8                                    |
|  Exemplo com ArrayList e forEach .....                                       | 8                                    |
|  Extras úteis para organizar listas .....                                    | 9                                    |
|  Parte 1: Enunciados com Scanner e ArrayList para entendimento inicial ..... | <b>Erro! Indicador não definido.</b> |
|  Enunciado 1 — Lista de compras simples .....                                | <b>Erro! Indicador não definido.</b> |
|  Enunciado 2 — Lista de presença .....                                       | <b>Erro! Indicador não definido.</b> |
|  Parte 2: tarefa individua – Agora é a sua vez: .....                        | 9                                    |
|  Tarefa 1 — Lista de tarefas do dia .....                                    | 9                                    |
|  Tarefa 2 — Cadastro de contatos .....                                       | 9                                    |
|  Tarefa 3 — Organizando uma lista de nomes .....                             | 9                                    |



# Apostila: ArrayList em Java — Métodos Essenciais



## Introdução

Antes de tudo, para usar ArrayList, precisamos importar a classe:

```
import java.util.ArrayList;
```

E criar uma lista:

```
ArrayList<String> nomes = new ArrayList<>();
```

## 1 add() — Adiciona elementos à lista



Adiciona um item ao final da lista.



Exemplo

```
import java.util.ArrayList;
```

```
public class ExemploAdd {  
    public static void main(String[] args) {  
        ArrayList<String> nomes = new ArrayList<>();  
        nomes.add("Gilmar");  
        nomes.add("Santos");  
        System.out.println(nomes);  
    }  
}
```



Saída

```
[Gilmar, Santos]
```

## 2 get(int index) — Acessa um elemento



Retorna o item na posição indicada.



Exemplo

```
ArrayList<String> nomes = new ArrayList<>();  
nomes.add("Gilmar");  
nomes.add("Santos");  
System.out.println(nomes.get(0));
```



Saída

```
Gilmar
```

## 3 set(int index, E elemento) — Altera um item



Substitui o valor em uma posição específica.



Exemplo

```
ArrayList<String> nomes = new ArrayList<>();
```

```
nomes.add("Gilmar");  
nomes.set(0, "João");  
System.out.println(nomes);
```

 Saída

[João]

## 4 remove(int index) — Remove por posição

 Remove o item da posição indicada.

 Exemplo

```
ArrayList<String> nomes = new ArrayList<>();  
nomes.add("Gilmar");  
nomes.add("Santos");  
nomes.remove(0);  
System.out.println(nomes);
```

 Saída

[Santos]

## 5 size() — Tamanho da lista

 Retorna quantos elementos existem na lista.

 Exemplo

```
ArrayList<String> nomes = new ArrayList<>();  
nomes.add("Gilmar");  
nomes.add("Santos");  
System.out.println(nomes.size());
```

 Saída:

2

## 6 clear() — Limpa a lista

 Remove todos os elementos da lista.

 Exemplo

```
ArrayList<String> nomes = new ArrayList<>();  
nomes.add("Gilmar");  
nomes.clear();  
System.out.println(nomes);
```

 Saída

[]

## contains(E elemento) — Verifica se existe

 Retorna true se o item estiver na lista.

 Exemplo

```
ArrayList<String> nomes = new ArrayList<>();  
nomes.add("Gilmar");  
System.out.println(nomes.contains("Gilmar"));
```

 Saída

True

## isEmpty() — Verifica se está vazia

 Retorna true se a lista não tiver elementos.

 Exemplo

```
ArrayList<String> nomes = new ArrayList<>();  
System.out.println(nomes.isEmpty());
```

 Saída:

True

## indexOf(E elemento) — Posição de um item

 Retorna o índice do item. Se não existir, retorna -1.

 Exemplo

```
ArrayList<String> nomes = new ArrayList<>();  
nomes.add("Gilmar");  
nomes.add("Santos");  
System.out.println(nomes.indexOf("Santos"));
```

 Saída:

1

## Métodos e recursos adicionais do ArrayList que valem ouro

### 1. replaceAll() — Substitui todos os elementos com base em uma regra

 Aplica uma função a cada elemento da lista.

### Exemplo

```
ArrayList<String> nomes = new ArrayList<>();
nomes.add("gilmar");
nomes.add("santos");
nomes.replaceAll(nome -> nome.toUpperCase());
System.out.println(nomes);
```

 Saída:

[GILMAR, SANTOS]

## 2. sort() — Ordena a lista

 Organiza os elementos em ordem crescente (alfabética ou numérica).

### Exemplo

```
import java.util.Collections;
```

```
ArrayList<String> nomes = new ArrayList<>();
nomes.add("Carlos");
nomes.add("Ana");
nomes.add("Bruno");
Collections.sort(nomes);
System.out.println(nomes);
```

 Saída:

[Ana, Bruno, Carlos]

## 3. reverse() — Inverte a ordem da lista

 Coloca os elementos na ordem inversa.

### Exemplo

```
Collections.reverse(nomes);
System.out.println(nomes);
```

 Saída:

[Carlos, Bruno, Ana]

## 4. subList(int fromIndex, int toIndex) — Pega uma parte da lista

 Cria uma nova lista com elementos entre dois índices.

### Exemplo

```
List<String> sub = nomes.subList(0, 2);
System.out.println(sub);
```

 Saída:

[Ana, Bruno]

## 5. forEach() — Itera com função lambda

 Executa uma ação para cada item da lista.

 Exemplo

```
nomes.forEach(nome -> System.out.println("Olá, " + nome));
```

 Saída:

Olá, Ana

Olá, Bruno

Olá, Carlos

## 6. removeIf() — Remove com condição

 Remove todos os elementos que atendem a uma regra.

 Exemplo

```
nomes.removeIf(nome -> nome.startsWith("A"));  
System.out.println(nomes);
```

 Saída:

[Bruno, Carlos]

## 7. equals() — Compara listas

 Verifica se duas listas são iguais (mesmos elementos e mesma ordem).

 Exemplo

```
ArrayList<String> lista1 = new ArrayList<>();  
ArrayList<String> lista2 = new ArrayList<>();  
lista1.add("A");  
lista2.add("A");  
System.out.println(lista1.equals(lista2));
```

 Saída:

True

## 8. forEach(System.out::println) — Imprime todos os elementos da lista

 O que é?

O método `forEach()` é usado para **executar uma ação para cada elemento da lista**.

Quando usamos `System.out::println`, estamos dizendo: “para cada item, imprima ele”.

Essa é uma forma moderna e limpa de percorrer listas, usando **expressões lambda**.

 Por que usar?

- Evita o uso de `for` tradicional

- Código mais curto e legível
- Ideal para exibir listas de forma rápida

#### Exemplo prático:

```
import java.util.ArrayList;

public class ExemploForEach {
    public static void main(String[] args) {
        ArrayList<String> nomes = new ArrayList<>();
        nomes.add("Gilmar");
        nomes.add("Santos");
        nomes.add("Ana");

        System.out.println("Lista de nomes:");
        nomes.forEach(System.out::println);
    }
}
```

#### Saída esperada

Lista de nomes:  
Gilmar  
Santos  
Ana

### o jeito tradicional com for:

```
for (String nome : nomes) {
    System.out.println(nome);
}
```

### for com índice — o clássico

#### Exemplo

```
ArrayList<String> nomes = new ArrayList<>();
nomes.add("Gilmar");
nomes.add("Santos");

for (int i = 0; i < nomes.size(); i++) {
    System.out.println(nomes.get(i));
}
```

#### Como funciona:

- i começa em 0
- nomes.size() retorna o total de elementos

- `nomes.get(i)` acessa o elemento na posição `i`

## DIREFENÇAS ENTRE FOR'S:

| Tipo de <code>for</code>          | Tem índice? | É moderno? | É simples? |
|-----------------------------------|-------------|------------|------------|
| <code>for</code> com índice       | ✓           | ✗          | ✓          |
| <code>for-each</code>             | ✗           | ✓          | ✓          |
| <code>forEach()</code> com lambda | ✗           | ✓ ✓        | ✓ ✓        |

## Dicas de nomes não mencionados nessa Apostila:

### 🧠 O que é uma expressão lambda em Java?

Uma **lambda** é uma forma curta de escrever uma **função anônima** — ou seja, uma função **sem nome**, que você pode passar como argumento para outro método.

Ela é usada principalmente para **executar ações em coleções**, como listas, de forma mais limpa e moderna.

📌 Estrutura básica de uma lambda

(parâmetros) -> { corpo da função }

**Mas se o corpo for simples, você pode até omitir as chaves:**

item -> `System.out.println(item)`

### 💡 Exemplo com `ArrayList` e `forEach`

```
ArrayList<String> nomes = new ArrayList<>();
nomes.add("Gilmar");
nomes.add("Santos");
```

```
nomes.forEach(nome -> System.out.println("Olá, " + nome));
```

📄 Saída:

Olá, Gilmar

Olá, Santos

## Extras úteis para organizar listas

- `Collections.shuffle(lista)` → embaralha os elementos
- `Collections.max(lista)` / `Collections.min(lista)` → encontra o maior ou menor valor
- `Collections.frequency(lista, elemento)` → conta quantas vezes um item aparece

## Parte 2: tarefa individua – Agora é a sua vez:

### Tarefa 1 — Lista de tarefas do dia

Crie um programa que permita ao usuário cadastrar tarefas do dia. Depois, pergunte se ele deseja remover alguma tarefa digitando o nome dela. Ao final, exiba a lista atualizada.

### Tarefa 2 — Cadastro de contatos

Crie um programa que permita ao usuário cadastrar nomes de contatos. Depois, pergunte se ele deseja buscar um nome específico. Informe se o nome está ou não na lista.

### Tarefa 3 — Organizando uma lista de nomes

Crie um programa que permita ao usuário digitar vários nomes. Ao final, organize a lista em ordem alfabética e exiba os nomes ordenados.